P4010R0
Funnel Shift Operations

Draft Proposal, 2026-02-09

Author:

Daniel Towner (Intel Corporation)

Audience:

SG6, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes adding funnel shift operations to the C++ standard library's `<bit>` header, providing both scalar and SIMD interfaces for this fundamental bit manipulation primitive.

Table of Contents

§ 1. Introduction

This paper proposes adding funnel shift operations to the C++ standard library. Funnel shifts concatenate two integer values, shift the concatenated result, and extract bits. This is a fundamental primitive bit operation with widespread use across many domains including cryptography, hashing, compression, and pseudo-random number generation.

Evidence for its widespread utility is that both scalar and SIMD forms of these instructions exist in all major architectures. Currently, programmers rely on the compiler recognizing manual bit manipulation patterns and optimizing them to these hardware instructions. However, it would be more useful and readable if programmers could directly specify the use of this operation through an explicit library function. This paper adds such a function to the `<bit>` header.

§ 1.1. Why "Funnel Shift"?

The name "funnel shift" describes the shape of the operation:

1. **Wide at top**: Two separate values (inputs)
2. **Concatenate**: They combine into a wider intermediate value (the wide part of the funnel)
3. **Shift & Extract**: Shift and extract bits from the concatenated value (narrowing back down—the narrow part of the funnel)

The data "funnels" from two inputs → wide intermediate → single output, like the shape of a physical funnel.

Hardware vendors use different terminology. Intel calls these "double shifts" (SHLD/SHRD) or "aligns" (PALIGNR, VALIGND), while ARM calls them "extract" (EXTR). However, the software ecosystem (LLVM, CUDA, Rust) has converged on "funnel shift" as the standard term, which this proposal adopts.

§ 1.2. How Funnel Shift Works

```
Funnel Shift Right Example: funnel_shift_right(0xABCD, 0x1234, 8)

Input values (16-bit each):
  high = 0xABCD = 1010101111001101
  low  = 0x1234 = 0001001000110100

Step 1: Concatenate (high in upper, or left-most, bits)

       0xABCD          0x1234
  ┌────────────────┬────────────────┐
  10101011110011010001001000110100

Step 2: Shift right by 8 bits

         0xAB          0xCD12
  ┌────────────────┬────────────────┐
  ........10101011110011010001001 0
```

```
Step 3: Extract right-most (lowest) 16 bits

      0xCD12
   ┌────────────────┐
  1100110100010010

Result: 0xCD12
```

Note: This diagram shows the conceptual semantics for understanding. The practical implementation typically uses `(low >> 8) | (high << 8)`, which computes the same result using smaller shifts, rather than a shift across the full width of the inputs.

§ 2. Motivation

§ 2.1. Use Cases

Funnel shifts are fundamental bit manipulation operations used extensively in:

1. **Hash functions**: MurmurHash, xxHash, and CityHash all use funnel shifts for bit mixing
2. **Cryptographic algorithms**: Block ciphers and stream ciphers frequently employ these operations
3. **Pseudorandom number generators**: The xorshift family and other PRNGs rely on funnel shifts
4. **Compression algorithms**: Bit-level data packing and extraction
5. **Bit-parallel algorithms**: String searching, pattern matching, and data processing

§ 2.2. Problems with Current Approach

Today, C++ programmers typically implement funnel shifts using a small sequence of shifts, ors, and a shift-count adjustment. While this can be made correct and efficient, it has several practical drawbacks:

- **Readability**: the intent ("funnel shift") is obscured by low-level mechanics (multiple shifts, ors, and `N - s` adjustments).
- **Error-prone**: small mistakes are easy to make and hard to review (off-by-one in the count, missing parentheses, and undefined behavior when shifting by the type width).
- **Edge cases**: programmers must decide (and consistently implement) what happens for `s == 0`, negative `s`, and `s` outside `[0, N)`, including whether and how to reduce the shift count.

- **Optimization variability**: recognition of these idioms and lowering to a single target instruction is not uniform across compilers, versions, and optimization settings.
- **SIMD portability**: there is no standard SIMD interface, so portable code must rely on platform-specific intrinsics (with differing names, semantics, and availability).

§ 3. Prior Art

To the best of our knowledge, funnel shifts have not been previously proposed to WG21. The P0553R4 proposal [P0553R4] (which became part of C++20) added bit manipulation functions including `rotl`, `rotr`, `countl_zero`, and `popcount` to the `<bit>` header, but did not include funnel shifts.

§ 3.1. Hardware Support

Both x86 and ARM provide native funnel shift instructions for scalar and SIMD operations, demonstrating the fundamental nature of this operation:

x86/x64:

- Scalar: SHLD/SHRD ("Shift Left/Right Double") — Intel 80386 (1985)
- SIMD byte-level: PALIGNR ("Packed Align Right") — SSSE3 (2006), VPALIGNR (AVX2, 2013)
- SIMD element-level: VALIGND/VALIGNQ ("Vector Align D/Q") — AVX-512F (2016)

ARM:

- Scalar: EXTR ("Extract Register") — ARMv8 AArch64 (2011)
- SIMD: VEXT ("Vector Extract") — NEON (2005), EXT (SVE)

§ 3.2. Software Support

All major software ecosystems provide funnel shift operations:

- **LLVM**: `fshl`/`fshr` intrinsics (widely used in LLVM IR) [LLVM-Funnel]
- **CUDA**: `__funnelshift_l`/`__funnelshift_r`/`__funnelshift_lc`/`__funnelshift_rc` intrinsics (available since compute capability 3.5) [CUDA-Intrinsics]

- **Rust**: `funnel_shl`/`funnel_shr` methods (unstable, tracking issue #145686) [Rust-Funnel]

4. Design

4.1. Design principles and edge cases

This proposal provides a portable intrinsic interface for funnel shifts, intended to map directly to existing target instructions.

In particular:

- **Preconditions**: the shift count is required to be within the natural in-range domain of common hardware funnel shift instructions.
- **No implicit shift-count reduction**: unlike `rotl`/`rotr`, these operations do not reduce the shift count modulo `N`.
- **Explicit identities**: `funnel_shift_right(high, low, 0)` returns `low` and `funnel_shift_left(high, low, 0)` returns `high`.

As discussed in [P3793R1], making these boundary cases explicit avoids the common pattern of scattering special-case guards around shift expressions, which can obscure intent.

4.1.1. Unsigned Integers Only

The operations are constrained to unsigned integer types for several reasons:

- **Bit pattern semantics**: Funnel shifts operate on bit patterns, viewing integers as sequences of bits without regard to sign representation. This is the natural domain for bit manipulation operations.
- **Well-defined shift semantics**: Right shifts on signed integers have implementation-defined behavior in C++ (arithmetic vs. logical shift). Constraining to unsigned integers ensures portable, predictable semantics across all platforms.
- **Consistency with existing facilities**: This design matches `rotl`/`rotr` in `<bit>`, which are also constrained to unsigned integer types for the same reasons.
- **Hardware alignment**: Native funnel shift instructions (x86 `SHLD`/`SHRD`, ARM `EXTR`) operate on bit patterns without sign interpretation, making unsigned integers the natural fit.

If sign-aware operations are needed, users can explicitly cast to unsigned types, perform the funnel shift, and cast back if appropriate for their use case.

§ 4.2. Parameter order

The proposed scalar and SIMD overloads use the parameter order `(high, low, s)`.

This ordering matches the specification model used throughout this paper: a conceptual 2N-bit value `combined` is formed by concatenating `high` and `low` from most-significant bits to least-significant bits (i.e. `high` provides bits `[N, 2N)` and `low` provides bits `[0, N)`). The result is then defined as extracting N bits from `combined`.

Placing the shift amount last follows established `<bit>`/`<simd>` conventions for shift-like operations and keeps common identities readable:

- `funnel_shift_right(high, low, 0)` returns `low`, and
- `funnel_shift_left(high, low, 0)` returns `high`.

While other interfaces (e.g. ISA mnemonics or compiler IR) may describe funnel shifts in terms of a "destination-like" operand and a "source-like" operand, the `(high, low, s)` order preserves the paper's high/low concatenation intent at the call site and still admits efficient lowering to existing hardware instructions.

§ 4.3. Naming

We propose `funnel_shift_left` and `funnel_shift_right`.

Hardware vendors use different names for this operation:

- **Intel/AMD**: "Double Shift" (SHLD/SHRD = Shift Left/Right Double)
- **ARM**: "Extract" (EXTR = Extract Register)
- **x86 SIMD**: "Align" (PALIGNR/VPALIGNR/VALIGND/Q = Packed Align Right, Vector Align D/Q)

However, the software ecosystem (LLVM, CUDA, Rust) has converged on funnel shift as the preferred term. The term accurately describes the operation and has become standard terminology in compiler and language implementations.

By using the full names:

- **Self-documenting**: Immediately clear to readers unfamiliar with the operation
- **Consistent with C++ conventions**: The `<bit>` library uses full words (`countl_zero`, `has_single_bit`, `popcount`, not abbreviations)
- **Industry precedent**: Both CUDA and Rust use unabbreviated "funnel" in their naming
- **Library philosophy**: Current C++ standard library favors clarity over brevity (e.g., `numeric_limits` not `num_lim`)

§ 4.4. Placement

The operations belong in the `<bit>` header alongside other bit manipulation functions:

- `rotl`, `rotr` (bit rotation)
- `countl_zero`, `countr_zero` (bit counting)
- `popcount` (population count)
- `has_single_bit`, `bit_width`, `bit_ceil`, `bit_floor` (bit utilities)

§ 4.5. Left and Right Variants

This paper proposes both `funnel_shift_left` and `funnel_shift_right` rather than a single directional operation.

The hardware evidence is strong to support this: major ISAs provide distinct operations for each direction (e.g. x86 `SHLD` vs `SHRD`), and compilers/language infrastructure model them as separate primitives (LLVM `fshl`/`fshr`, CUDA `__funnelshift_l*`/`__funnelshift_r*`). While one direction can be expressed in terms of the other by swapping operands and transforming the shift count, providing both preserves intent and enables direct, predictable mapping to the best target instruction.

§ 4.6. Relationship to Rotation

Funnel shifts generalize bit rotation operations. When both inputs are identical, funnel shift reduces to rotation:

```
// These are equivalent for valid shift counts:
auto result1 = std::rotl(x, s);
auto result2 = std::funnel_shift_left(x, x, s);
```

```
// Similarly:
auto result3 = std::rotr(x, s);
auto result4 = std::funnel_shift_right(x, x, s);
```

§ 4.7. Scalar Interface

```
namespace std {
  template<class T, class S>
    constexpr T funnel_shift_left(T high, T low, S s) noexcept;

  template<class T, class S>
    constexpr T funnel_shift_right(T high, T low, S s) noexcept;
}
```

Constraints:

- `T` is an unsigned integer type.
- `S` is an integral type.

Preconditions: `0 <= s < N`, where `N` is `numeric_limits<T>::digits`.

Semantics: Let `N` be `numeric_limits<T>::digits` and let `r` be `static_cast<unsigned>(s)`.

- `funnel_shift_right(high, low, s)` is equivalent to `(low >> r) | (high << ((N - r) % N))`.
- `funnel_shift_left(high, low, s)` is equivalent to `(high << r) | (low >> ((N - r) % N))`.

§ 4.8. SIMD Interface

We propose SIMD overloads to be included in the initial specification, providing both uniform-shift and per-element-shift variants:

```
namespace std {
  // Per-element shift amounts
  template<simd-vec-type V0, simd-vec-type V1>
    constexpr V0 funnel_shift_left(const V0& high, const V0& low, const V1&
```

```
  template<simd-vec-type V0, simd-vec-type V1>
    constexpr V0 funnel_shift_right(const V0& high, const V0& low, const V1

  // Uniform shift amount
  template<simd-vec-type V, class S>
    constexpr V funnel_shift_left(const V& high, const V& low, S s) noexcep
  template<simd-vec-type V, class S>
    constexpr V funnel_shift_right(const V& high, const V& low, S s) noexce
}
```

These operations apply the scalar funnel shift element-wise, matching the design of `rotl`/`rotr` and shift operators in SIMD. Including SIMD support from the outset is justified by several factors:

1. **Extensive hardware precedent**: SIMD funnel shift instructions have existed as long as their scalar counterparts. ARM NEON's `VEXT` instruction (vector extract) has been available since 2005, and x86's `PALIGNR` (packed align right) since SSSE3 in 2006. The AVX-512 element-level funnel shifts (`VALIGND`/`VALIGNQ`) have been available since 2016. This 18+ year history demonstrates that SIMD funnel shifts are not experimental but fundamental operations with proven utility.

2. **Consistency with existing SIMD bit operations**: P0214 (Data-Parallel Vector Types & Operations) includes SIMD versions of `rotl`/`rotr` in its initial specification. Since funnel shifts are the generalization of rotations (as shown in § 4.6 Relationship to Rotation), excluding SIMD funnel shifts while including SIMD rotations would create an artificial inconsistency in the `<bit>` and `<simd>` interface.

3. **No design tensions between scalar and SIMD**: The scalar and SIMD interfaces are natural extensions of each other with identical semantics applied element-wise. There are no unresolved design questions or competing approaches. Both forms follow the same parameter order and the same constraints on unsigned integer types. The two variants (uniform-shift and per-element-shift) directly parallel the existing patterns for SIMD shift and rotation operations.

4. **Implementation experience**: LLVM's `fshl`/`fshr` intrinsics work identically for both scalar and vector types, demonstrating that the unified design is well-understood and implementable. Compilers already optimize vector funnel shift patterns to the appropriate SIMD instructions (`VEXT`, `PALIGNR`, `VALIGND`) just as they optimize scalar patterns to `SHLD`/`SHRD`/`EXTR`, when the shift count is within the natural in-range domain of those instructions.

5. **Avoid API churn**: Splitting scalar and SIMD into separate proposals would require two rounds of standardization for what is conceptually a single operation. This delays adoption and creates a temporary period where scalar funnel shifts are available but SIMD versions require platform-specific intrinsics, defeating the portability goal.

6. **Real-world SIMD usage**: Vectorized hashing, parallel bit stream processing, and SIMD-accelerated cryptographic operations all benefit from SIMD funnel shifts. For example, vectorized implementations of hash functions that process multiple independent streams in parallel require the same bit manipulation primitives as their scalar counterparts.

§ 5. Examples

§ 5.1. Hash Mixing

```
#include <bit>
#include <cstdint>

uint32_t mix(uint32_t x, uint32_t y) {
    return std::funnel_shift_right(x, y, 17);
}
```

§ 5.2. Bit Field Extraction Across Word Boundaries

```
// Extract bits from a bit stream stored in an array
uint64_t extract_bits(const uint64_t data[], size_t bit_offset) {
    size_t word_idx = bit_offset / 64;
    size_t bit_in_word = bit_offset % 64;

    if (bit_in_word == 0) {
        return data[word_idx];
    }

    return std::funnel_shift_right(
        data[word_idx + 1],
        data[word_idx],
        bit_in_word
    );
}
```

§ 6. Implementation Experience

The proposed operations are straightforward to implement and have been proven in practice.

§ 6.1. Reference Implementation

The practical implementation uses only N-bit operations, avoiding the need for 2N-bit types:

```
template<class T, class S>
constexpr T funnel_shift_right(T high, T low, S s) noexcept {
    constexpr auto N = std::numeric_limits<T>::digits;
    // Preconditions: 0 <= s < N
    const auto r = static_cast<unsigned>(s);
    if (r == 0) return low;
    return (low >> r) | (high << (N - r));
}

template<class T, class S>
constexpr T funnel_shift_left(T high, T low, S s) noexcept {
    constexpr auto N = std::numeric_limits<T>::digits;
    // Preconditions: 0 <= s < N
    const auto r = static_cast<unsigned>(s);
    if (r == 0) return high;
    return (high << r) | (low >> (N - r));
}
```

This implementation is mathematically equivalent to the conceptual description (concatenate, shift, extract) but works entirely in the input type's width.

§ 6.2. Compiler Codegen

Modern compilers already optimize manual funnel shift patterns to native instructions:

```
// This pattern:
uint32_t manual = (low >> shift) | (high << (32 - shift));

// Compiles to (x86):
//   shrdl %cl, %esi, %eax    // Single instruction
```

This demonstrates that the proposed functions map directly to efficient hardware implementations when the shift count is in range.

7. Proposed Wording

7.1. Header `<bit>` synopsis [bit.syn]

Add to the synopsis:

```
namespace std {
  // ...existing declarations...

  // [bit.funnel], funnel shifts
  template<class T, class S>
    constexpr T funnel_shift_left(T high, T low, S s) noexcept;
  template<class T, class S>
    constexpr T funnel_shift_right(T high, T low, S s) noexcept;
}
```

7.2. Funnel shifts [bit.funnel]

```
template<class T, class S> constexpr T funnel_shift_right(T high, T low, S
```

Let:

- `N` be `numeric_limits<T>::digits`, and
- `r` be `static_cast<unsigned>(s)`.

Constraints:

- `T` is an unsigned integer type ([basic.fundamental]).
- `S` models integral ([concepts]).

Preconditions: `0 <= s < N`

Returns: `(low >> r) | (high << ((N - r) % N))`.

Remarks: `funnel_shift_right(high, low, 0)` returns `low`.

```
template<class T, class S> constexpr T funnel_shift_left(T high, T low, S s
```

Let:

- `N` be `numeric_limits<T>::digits`, and
- `r` be `static_cast<unsigned>(s)`.

Constraints:

- `T` is an unsigned integer type ([basic.fundamental]).
- `S` models integral ([concepts]).

Preconditions: `0 <= s < N`

Returns: `(high << r) | (low >> ((N - r) % N))`.

Remarks: `funnel_shift_left(high, low, 0)` returns `high`.

§ 7.3. [simd.syn]

In [simd.syn], add the following declarations to the `[simd.bit]` section after `rotr`:

```
  template
    constexpr V0 rotl(const V0& v, const V1& s) noexcept;
  template
    constexpr V  rotl(const V& v, int s) noexcept;

  template
    constexpr V0 rotr(const V0& v, const V1& s) noexcept;
  template
    constexpr V  rotr(const V& v, int s) noexcept;


  template
    constexpr V0 funnel_shift_left(const V0& high, const V0& low, const V1&
  template
    constexpr V0 funnel_shift_right(const V0& high, const V0& low, const V1
```

```
  template
    constexpr V  funnel_shift_left(const V& high, const V& low, S s) noexce
  template
    constexpr V  funnel_shift_right(const V& high, const V& low, S s) noexc
```

7.3.1. [simd.bit]

In [simd.bit], insert the following after the `rotl`/`rotr` specifications:

```
template
  constexpr V0 funnel_shift_left(const V0& high, const V0& low, const V1& s
template
  constexpr V0 funnel_shift_right(const V0& high, const V0& low, const V1&
```

Constraints:

- The type `V0::value_type` is an unsigned integer type ([basic.fundamental]),
- the type `V1::value_type` models integral,
- `V0::size() == V1::size()` is `true`, and
- `sizeof(typename V0::value_type) == sizeof(typename V1::value_type)` is `true`.

Returns: A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(high[i], low[i], s[i])` for all i in the range `[0, V0::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

```
template
  constexpr V funnel_shift_left(const V& high, const V& low, S s) noexcept
template
  constexpr V funnel_shift_right(const V& high, const V& low, S s) noexcep
```

Constraints:

- The type `V::value_type` is an unsigned integer type ([basic.fundamental]), and
- `S` models integral ([concepts]).

Returns: A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(high[i], low[i], s)` for all i in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

7.4. Feature test macro

Add to [version.syn]:

```
#define __cpp_lib_funnel_shift 2026XXL // P4010, also in <bit>
```

8. Acknowledgements

Thanks to Jan Schultke for detailed feedback on the wording and on shift-count semantics, and for discussion of the tradeoffs between "portable intrinsic" behavior and always-defined behavior for bit manipulation facilities.

References

Informative References

[CUDA-Intrinsics]
Nicholas Wilt; NVIDIA. *The CUDA Handbook (and CUDA intrinsic documentation for __funnelshift_*)*. URL: https://developer.nvidia.com/cuda-toolkit

[LLVM-Funnel]
LLVM Project. *LLVM Language Reference Manual: llvm.fshl and llvm.fshr intrinsics*. URL: https://llvm.org/docs/LangRef.html#llvm-fshl-intrinsic

[P0553R4]
Jens Maurer. *P0553R4: Bit operations*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0553r4.html

[P3793R1]
Brian Bi; Jan Schultke. *P3793R1: Better shifting*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3793r1.html

[Rust-Funnel]
Rust Project. *Tracking Issue for Integer Funnel Shifts (Rust issue #145686)*. URL: https://github.com/rust-lang/rust/issues/145686